

System Request: Production-Grade Real-Time Chat System

1. Objective

Build a scalable, production-ready chat system that supports real-time messaging, presence features, and horizontal scaling. The system must handle stateful WebSocket connections in a stateless cloud environment while guaranteeing message ordering, delivery semantics, and resilience to failures.

2. Functional Requirements

Core Features

- User authentication using JWT (access + refresh tokens)
- Create, list, and join named chat rooms
- Real-time messaging over WebSockets
- Typing indicators (broadcast within room)
- Message persistence (store all messages with timestamps)
- Read receipts (mark messages as read per user)

Non-Functional / Advanced (Mandatory)

- **Message ordering guarantee** – messages in a room must be delivered in the exact order they were sent (using sequence numbers or timestamps + conflict resolution).
- **Reconnection logic** – clients can resume a WebSocket session without missing messages (session ID + last received sequence number).
- **Rate limiting** – anti-spam limits per user (e.g., 10 messages/second).
- **Message queue** – incoming messages go through an async queue before persistence and broadcast (decouple ingestion from processing).
- **Horizontal scaling awareness** – multiple backend instances must synchronize room broadcasts (no single-node dependency).

3. System Design Constraints

- **API layer** – FastAPI (REST for auth, rooms; WebSocket for real-time).
- **WebSocket manager** – manage connections, room subscriptions, and session resumption.
- **Service layer** – business logic (send message, typing, receipts).
- **Repository layer** – database access abstraction.

- **Event system** – publish/subscribe pattern for cross-instance communication.

4. Required Technologies

Component	Technology (minimum)
API & WS	FastAPI
Database	PostgreSQL
Cache / PubSub	Redis (Pub/Sub for multi-instance sync)
ORM / Migrations	SQLAlchemy + Alembic
Message Queue	Optional but strongly encouraged: RabbitMQ or Kafka
Task Queue	Optional: Celery (for async processing)
Containerization	Docker + Docker Compose

5. Dockerization Requirements

Deliver a `docker-compose.yml` with:

- FastAPI service
- PostgreSQL (with volume and health check)
- Redis (with health check)
- Proper networking (internal bridge)
- Environment variables via `.env`
- Health checks for all services
- Optional: RabbitMQ/Kafka container

6. Deliverables

- Fully working code (Git repository)
- `README.md` with:
 - Setup instructions (Docker Compose up)
 - API & WebSocket protocol documentation
 - Explanation of how message ordering, reconnect logic, and cross-instance sync work
- Postman / WebSocket client collection (or simple HTML/JS demo client)

7. Success Criteria

- Two or more backend instances can run behind a load balancer; messages broadcast correctly to all instances.
- A client reconnects after network failure and receives missed messages without duplicates.
- Message ordering is preserved even under high load.
- Rate-limited users receive appropriate error responses.
- All services pass `docker-compose up --health-check` .